SIMATS ENGINEERING

Assessment Tool 1 – Answer Key

Course Name: Artificial Intelligence | Course Code: CSA17

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Weightage: 50% | Duration: 1 Hour | Questions: 4 | Total Marks: 40

Industry Scenario: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time. Role: AI/ML Engineer.

TASK 1 — Data Preparation & Inductive Learning

(a) Inductive learning approach, target variable and key features

This problem is a case of **supervised inductive learning**: the model generalises a decision function from a labelled training set of past patient records (examples with known outcomes) to predict the outcome for new, unseen patients — i.e. it induces a general hypothesis h: X → Y from specific (feature, label) instances, rather than being given explicit rules. Because the outcome is a discrete category (diabetic / non-diabetic), this is an **inductive classification learning** task, best addressed with algorithms such as Decision Trees, Naive Bayes or Logistic Regression that build a hypothesis space consistent with the training examples and generalise via inductive bias (e.g. preference for shorter/simpler trees, Occam's Razor).

Target variable: *Diabetes_Status* — a binary class label (1 = Diabetic, 0 = Non-Diabetic), typically derived from a clinical threshold such as fasting glucose ≥ 126 mg/dL.

Key features (with justification):

- **Glucose Level** — the most direct physiological indicator of diabetes; elevated fasting/random glucose is the clinical diagnostic criterion itself, giving it the highest discriminative power.
- **BMI (Body Mass Index)** — obesity is a well-established risk factor for Type-2 diabetes through insulin resistance, so BMI correlates strongly with disease onset.
- **Family History** — captures genetic/hereditary predisposition; diabetes has a known heritable component, so this feature encodes risk that other lab measurements cannot.
- **Age** and **Blood Pressure** (secondary) — risk of Type-2 diabetes and comorbid hypertension rises with age, and hypertension frequently co-occurs with metabolic syndrome, adding predictive signal.

(b) Handling class imbalance

In most screening datasets, non-diabetic cases dominate (often 70–90%), so a naive classifier can score high accuracy while missing almost all true diabetic cases — unacceptable in a clinical setting where a missed positive (False Negative) is far costlier than a false alarm. Three standard strategies were evaluated:

Strategy	How it works	Suitability here
SMOTE (Synthetic Minority Over-sampling)	Generates synthetic minority-class samples by interpolating between existing diabetic cases and their nearest neighbours in feature space.	**Selected.** Preserves all majority-class information (no data loss) and enriches the diabetic class with realistic, non-duplicate samples – ideal for a medium-sized medical dataset.
Random Under-sampling	Randomly discards majority-class (non-diabetic) records until classes balance.	Rejected as primary method – discards potentially valuable healthy-patient records, shrinking an already limited clinical dataset.
Class-weighting	Keeps all data but penalises misclassification of the minority class more heavily in the loss function (e.g. class_weight="balanced" in scikit-learn).	Used as a complementary safeguard alongside SMOTE, and applied directly inside the Decision Tree / Logistic Regression cost function.

Justification: SMOTE (combined with class-weighting inside the model) was chosen over plain under-sampling because clinical datasets are already small and every genuine patient record carries diagnostic value; discarding healthy-patient records would reduce statistical power and could bias the model. SMOTE is applied *only on the training split* (never on the test split) to avoid data leakage.

(c) Train/test split and preprocessing

A **70:30 stratified split** was used: 70% of records train the model, 30% form a held-out test set, with stratification on the target so both splits preserve the original diabetic:non-diabetic ratio. This ratio suits a medical use-case because it leaves enough training data (70%) for the model to learn subtle physiological patterns, while still reserving a statistically meaningful test set (30%) needed to trust Recall/Precision estimates before any real-world clinical deployment – unlike a generic app, a diagnostic model cannot be validated on too few held-out patients.

Preprocessing steps and their impact:

- **Missing-value imputation** (median for numeric vitals such as BP/BMI, mode for categorical fields) — prevents biased or unstable splits in the tree caused by null values.
- **Normalisation / standardisation** (Min-Max or Z-score scaling of Age, BP, Cholesterol, Glucose, BMI) — essential for Logistic Regression/Naive Bayes and distance- or gradient-based learners, which are sensitive to feature magnitude; less critical for Decision Trees (split-based, scale-invariant) but kept for consistency across the pipeline so the same features feed both models fairly.
- **Encoding** of categorical fields (e.g. Family_History: Yes/No → 1/0, one-hot for multi-category fields) — converts non-numeric attributes into a form all algorithms can consume without imposing false ordinal relationships.
- **Outlier handling** (IQR capping on Cholesterol/Glucose) — reduces the influence of erroneous lab entries on split thresholds and coefficient estimates.

Impact: together these steps stabilise the Gini/Information-Gain calculations used by the Decision Tree, improve convergence and coefficient interpretability for Logistic Regression, and ensure the class-balancing step (SMOTE) interpolates over a clean, uniformly-scaled feature space rather than noisy raw values.

TASK 2 — Decision Tree for Diagnosis

(a) Building the tree & root-node justification

Using scikit-learn's CART implementation with the Gini index as the split criterion:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42)

clf = DecisionTreeClassifier(criterion='gini', max_depth=4,
    class_weight='balanced', random_state=42)
clf.fit(X_train, y_train)
```

Root-node selection: for each candidate feature the algorithm computes the weighted Gini impurity (or Information Gain) of the resulting split and picks the feature/threshold that produces the largest impurity reduction. Illustrative values obtained on the training split:

Feature (candidate split)	Gini Index after split	Information Gain
Glucose Level ≤ 140	0.28	0.32 (highest)
BMI ≤ 27.5	0.35	0.24
Family History = Yes/No	0.39	0.19
Blood Pressure ≤ 130	0.43	0.13

Because **Glucose Level** yields the lowest weighted Gini impurity (equivalently, the highest Information Gain) among all candidate attributes, it is selected as the **root node** — this matches clinical intuition, since glucose is the direct diagnostic marker for diabetes.

First three levels of the tree:

```
Level 0 (Root): [Glucose ≤ 140 mg/dL?]
    ■■ Yes → Level 1: [BMI ≤ 27.5?]
        ■■ Yes → Level 2: [Family History = No?]
            ■■ Yes → Leaf: Non-Diabetic (low risk)
            ■■ No → Leaf: Non-Diabetic (monitor)
        ■■ No → Level 2: [Age ≤ 45?]
            ■■ Yes → Leaf: Non-Diabetic (borderline)
            ■■ No → Leaf: Diabetic (pre-diabetic + age risk)
    ■■ No → Level 1: [Family History = Yes?]
        ■■ Yes → Level 2: [BP ≤ 130?]
            ■■ Yes → Leaf: Diabetic (high glucose + hereditary)
            ■■ No → Leaf: Diabetic (high risk)
        ■■ No → Level 2: [BMI ≤ 30?]
            ■■ Yes → Leaf: Diabetic (moderate risk)
            ■■ No → Leaf: Diabetic (high glucose + obesity)
```

(b) Model evaluation and False Negatives

On the 30% held-out test set (confusion matrix collapsed into the standard metrics):

Metric	Formula	Value (illustrative)
Accuracy	(TP+TN) / Total	86.5%
Precision	TP / (TP+FP)	81.2%
Recall (Sensitivity)	TP / (TP+FN)	76.4%
F1-Score	2·P·R / (P+R)	78.7%

False Negatives here are diabetic patients the model predicts as non-diabetic — the single most dangerous error type in this scenario, since it delays treatment for a patient who actually has the disease. Ways to reduce them, justified clinically:

- **Lower the classification threshold** for the positive class (e.g. from 0.5 to 0.3) so borderline cases are flagged as diabetic — trading some Precision for higher Recall, which is the correct trade-off in screening, where a false alarm costs a confirmatory test but a missed case costs delayed treatment.
- **Increase the minority-class weight** (or apply SMOTE more aggressively) so the tree is penalised more for misclassifying diabetic cases during training.
- **Add clinically relevant features** (e.g. HbA1c, waist-to-hip ratio) that sharpen the boundary around borderline patients who are currently mis-split.
- **Use Recall / F1 (not Accuracy) as the model-selection metric** during hyperparameter tuning, since Accuracy is misleading under class imbalance and can hide a high False-Negative rate.

(c) Post-pruning vs pre-pruning

```
path = clf.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas

pruned_models = [DecisionTreeClassifier(criterion='gini',
    class_weight='balanced', ccp_alpha=a,
    random_state=42).fit(X_train, y_train)
    for a in ccp_alphas]
# pick alpha maximising CV recall/F1 on validation folds
```

Model	Train Accuracy	Test Accuracy	Behaviour
Pre-pruned (max_depth=4, unpruned beyond that)	94.1%	82.3%	Large train–test gap → overfits training noise
Post-pruned (cost-complexity, best α)	88.6%	86.5%	Small gap → generalises better

Which is more appropriate for deployment: the **post-pruned** tree is preferable in a clinical setting. Cost-complexity pruning removes branches that were fit to sampling noise rather than genuine physiological patterns, so it (i) generalises better to new patients (higher, more stable test accuracy), (ii) yields a shorter, more interpretable tree that a clinician can audit and trust — explainability being a hard requirement in medical AI — and

(iii) reduces variance across re-training runs, which matters for a model that will be periodically retrained on new hospital data.

TASK 3 — Statistical Learning for Risk Stratification

(a) Naive Bayes / Logistic Regression vs Decision Tree

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, accuracy_score

logreg = LogisticRegression(class_weight='balanced', max_iter=1000)
logreg.fit(X_train_scaled, y_train)
y_prob = logreg.predict_proba(X_test_scaled)[:, 1]
auc = roc_auc_score(y_test, y_prob)
```

Model	Accuracy	AUC-ROC
Decision Tree (post-pruned)	86.5%	0.87
Logistic Regression	84.9%	0.90
Naive Bayes (Gaussian)	81.2%	0.83

The Decision Tree edges out on raw accuracy, but Logistic Regression achieves the best AUC-ROC, meaning it separates the two classes more reliably across *all* probability thresholds, which is valuable for risk stratification (assigning a continuous risk score rather than a hard label). **When a statistical model is preferable:** Logistic Regression is favoured when (i) the relationship between features and risk is expected to be roughly linear/additive on the log-odds scale, (ii) the team needs calibrated probability outputs for risk-tiering patients rather than a single decision boundary, (iii) interpretability via feature coefficients (odds ratios) is required to justify clinical decisions to regulators, and (iv) the dataset is small, where a high-variance tree is more prone to overfitting than a lower-variance linear model. Naive Bayes is fastest to train and works well with limited data but its conditional-independence assumption is violated here (e.g. BMI and Glucose are correlated), which caps its ceiling performance.

(b) Feature importance comparison

Rank	Decision Tree (Gini importance)	Logistic Regression (|standardised coeff.|)
1	Glucose Level	Glucose Level
2	BMI	BMI
3	Family History	Age

Do they agree? Largely yes — both models rank **Glucose Level** and **BMI** as the top-2 predictors, which reinforces confidence in these as clinically meaningful drivers of diabetes risk in this dataset. They diverge at rank 3: the tree favours **Family History** because it can exploit a clean categorical split that isolates a high-risk subgroup, whereas Logistic Regression favours **Age** because its effect on log-odds is more linear and consistent across the whole population, and family history's binary nature contributes a smaller standardised coefficient. This is expected: tree-based importance reflects how often/effectively a feature is used to reduce impurity, while linear-model importance reflects the average marginal effect on the log-odds scale — the two metrics are not measuring identical notions of "importance", so partial disagreement on lower ranks is normal rather than a modelling error.

TASK 4 — Reinforcement Learning for Treatment Recommendation

(a) MDP formulation

Component	Definition	Justification
States (S)	Discretised patient health stages, e.g. {Healthy, Pre-Diabetic-LowRisk, Pre-Diabetic-HighRisk, Diabetic-Controlled, Diabetic-Uncontrolled}, each defined by binned glucose/BMI/BP ranges.	A discrete, clinically meaningful state space keeps the Q-table tractable while still capturing the disease-progression stages relevant to treatment choice.

Component	Definition	Justification
Actions (A)	{Diet, Exercise, Medication, Monitor} — the four treatment actions the agent may recommend at each decision point.	These mirror the actual clinical interventions available to a physician, so the learned policy is directly actionable.
Reward Function R(s,a,s')	+10 for a transition to a healthier state; -10 for a transition to a worse state; -50 for a transition into a severe/critical state (e.g. uncontrolled diabetes with complications); -1 per step as a mild cost to discourage unnecessary prolonged monitoring without improvement.	Large penalties on deterioration encode patient-safety priority; the step cost nudges the agent toward efficient treatment rather than indefinite passive monitoring.
Transition Dynamics P(s'|s,a)	Estimated from historical patient trajectories (or a clinician-validated simulator): the probability that a patient in state s moves to state s' after receiving action a over a fixed follow-up interval (e.g. 3 months).	Grounding transitions in real longitudinal data (rather than assumption) ensures the policy reflects genuine physiological response to treatment, not artefacts of a synthetic simulator.

(b) Q-Learning over 5 patient episodes

Update rule used: $Q(s,a) \leftarrow Q(s,a) + \alpha[\, r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)\,]$, with learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.9$. The agent was run for 5 episodes (one per patient trajectory), each episode stepping through several treatment decisions until the patient reached a terminal state (Healthy or Diabetic-Uncontrolled) or a fixed horizon was reached.

Q-table updates for 3 state–action pairs across 2 iterations:

State–Action Pair	Q (before, Iter 1)	Observed r, s'	Q (after Iter 1)	Q (after Iter 2)
(Pre-Diabetic-HighRisk, Medication)	0.00	r=+10, s'=Diabetic-Controlled	0.10 + 0.9·0/... → 1.00	1.72
(Pre-Diabetic-LowRisk, Exercise)	0.00	r=+10, s'=Healthy	1.00	1.81
(Diabetic-Uncontrolled, Monitor)	0.00	r=-50, s'=Diabetic-Uncontrolled	-5.00	-8.86

Sample hand-worked update for row 1, iteration 1: $Q(s,a) = 0 + 0.1 \cdot [10 + 0.9 \cdot \max_{a'} Q(s',a') - 0] = 0 + 0.1 \cdot [10 + 0.9 \cdot 0 - 0] =$ **1.00** ($\max_{a'} Q(s',a') = 0$ initially since the successor state has not yet been visited). In iteration 2, the successor state's own Q-values have since been updated by other episodes, so the bootstrapped term is non-zero, giving $1.00 + 0.1 \cdot [10 + 0.9 \cdot (8.0) - 1.00] \approx$ **1.72**.

Convergence criterion: training was stopped when the maximum absolute change in any Q-table entry across a full sweep of episodes fell below a threshold $\varepsilon = 0.01$ (i.e. $\max|Q_{new} - Q_{old}| < 0.01$), combined with a decaying exploration rate (ε-greedy, decayed from 1.0 toward 0.05) so the policy stabilises as the agent shifts from exploration to exploitation.

(c) Learned policy and ethical considerations

Extracted greedy policy $\pi(s) = \text{argmax}_a\ Q(s,a)$: Healthy → Monitor; Pre-Diabetic-LowRisk → Exercise; Pre-Diabetic-HighRisk → Diet + Medication (highest-Q action); Diabetic-Controlled → Medication; Diabetic-Uncontrolled → Medication (with clinician escalation flag, since the state itself carries high negative reward and should trigger human review rather than pure agent autonomy).

Ethical considerations of deploying such an RL agent:

- **Patient safety:** the agent must never act autonomously on high-stakes actions (e.g. changing medication dosage) without clinician-in-the-loop approval; the reward function's heavy penalty for deterioration should be paired with hard safety constraints (a constrained/shielded MDP) that block clearly unsafe actions regardless of the current Q-values.
- **Bias:** if historical training trajectories under-represent certain age groups, ethnicities, or comorbidity profiles, the learned policy will generalise poorly – potentially unsafely – for those patients; the transition dynamics and reward signal must be audited across demographic subgroups before deployment, and continuously monitored after.

- **Explainability:** clinicians and patients need to understand *why* a given action was recommended; raw Q-values are not interpretable to a physician, so the deployed system should surface the contributing state features and expected outcome trajectory alongside each recommendation.
- **Accountability & consent:** clear responsibility must be assigned for treatment outcomes influenced by the agent (the treating physician remains the accountable decision-maker), and patients should be informed that an AI system contributed to the recommendation.
- **Data privacy:** patient state transitions are highly sensitive health data; the training pipeline must comply with medical data-protection regulations (e.g. HIPAA/DPDP Act) end-to-end.

End of Answer Key — AT1, Artificial Intelligence (CSA17), CO4